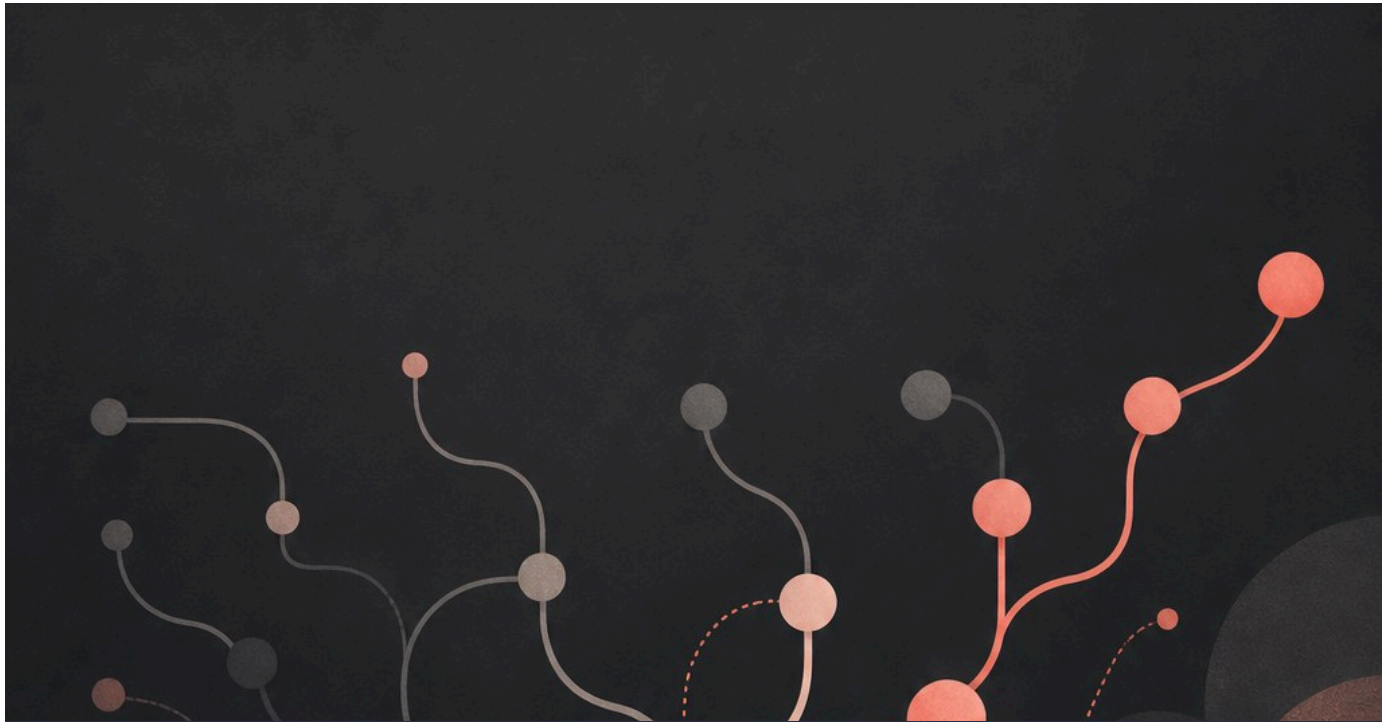




FORMATION INFORMATIQUE

Git

Garder l'historique de son code, collaborer
sans se marcher dessus.

DanielCraft - Livre debutant

Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

1. Chapitre 1 - Salut, c'est quoi Git ?

- [A quoi ca sert ?](#)
- [Git vs GitHub](#)
- [Image mentale](#)
- [Ce que tu vas savoir faire](#)
- [Comment lire ce livre](#)
- [Mythe a casser](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)

2. Chapitre 2 - Installer Git

- [Windows](#)
- [Premiere config \(obligatoire\)](#)
- [Ou taper les commandes ?](#)
- [Editeur de message](#)
- [Aide integree](#)
- [A toi](#)
- [Erreur classique](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)

3. Chapitre 3 - Ton premier depot

- [Creer le dossier](#)
- [Initialiser Git](#)
- [Premier fichier](#)
- [Voir l'etat](#)
- [Zones a retenir](#)
- [git init dans un projet existant](#)
- [A toi](#)
- [Erreur classique](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)
- [Trois zones, encore une fois](#)

4. Chapitre 4 - add et commit (la photo)

- [Ajouter un fichier](#)
- [Ajouter plusieurs fichiers](#)
- [Commit](#)
- [Voir que ca a marche](#)
- [Modifier encore](#)
- [Schema mental](#)
- [Amend \(aperçu, avec prudence\)](#)
- [A toi](#)
- [En vrai, sur le terrain](#)

- [Mini defi](#)

5. [Chapitre 5 - Lire l'historique](#)

- [log simple](#)
- [log compact](#)
- [Voir un fichier a travers le temps](#)
- [diff : qu'est-ce qui a change ?](#)
- [show](#)
- [Checkout d'un fichier \(aperçu\)](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)

6. [Chapitre 6 - Les branches](#)

- [Voir les branches](#)
- [Creer et changer de branche](#)
- [Travailler sur la branche](#)
- [Revenir sur main](#)
- [A quoi ca sert vraiment ?](#)
- [Renommer / supprimer](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)

7. [Chapitre 7 - Fusionner \(merge\)](#)

- [Scenario](#)
- [Si tout va bien](#)
- [Fast-forward, c'est quoi ?](#)
- [Merge avec message](#)
- [Apres le merge](#)
- [Quand ne pas merger directement ?](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)
- [Merge vs rebase \(aperçu\)](#)

8. [Chapitre 8 - Les conflits](#)

- [Comment ca arrive](#)
- [A quoi ca ressemble](#)
- [Resolving](#)
- [Annuler un merge en cours](#)
- [Eviter les conflits \(un peu\)](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)

9. [Chapitre 9 - .gitignore](#)

- [Creer le fichier](#)

- [Verifier](#)
- [Deja suivi par erreur ?](#)
- [Templates](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)
- [Patterns utiles](#)

10. [Chapitre 10 - GitHub, c'est quoi ?](#)

- [Compte](#)
- [Nouveau depot en ligne](#)
- [Lier ton depot local](#)
- [Voir les remotes](#)
- [SSH ou HTTPS ?](#)
- [README.md](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)

11. [Chapitre 11 - clone, push, pull](#)

- [clone](#)
- [push](#)
- [pull](#)
- [fetch \(aperçu\)](#)
- [Schema](#)
- [Erreurs frequentes](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)

12. [Chapitre 12 - De bons messages de commit](#)

- [Regle simple](#)
- [Oui](#)
- [Non](#)
- [Corps optionnel](#)
- [Conventional Commits \(aperçu\)](#)
- [Une idee = un commit ?](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)
- [Exemple de fil propre](#)

13. [Chapitre 13 - Mini-projet : carnet versionne](#)

- [But](#)
- [Etapas](#)
- [Criteres de reussite](#)
- [Variante avancee](#)

- [En vrai, sur le terrain](#)
- [Mini defi](#)

14. [Chapitre 14 - Ce qu'il faut retenir \(bases\)](#)

- [Boucle quotidienne](#)
- [Commandes vitales](#)
- [Zones](#)
- [Habitudes](#)
- [Erreurs classiques](#)
- [Suite immediate](#)
- [Mini check](#)
- [Aide-memoire express](#)

15. [Chapitre 15 - Stash et annuler \(avec prudence\)](#)

- [stash](#)
- [restore \(annuler des modifs locales\)](#)
- [restore --staged](#)
- [reset --soft / --mixed \(aperçu\)](#)
- [reset --hard \(danger\)](#)
- [revert \(plus sûr en équipe\)](#)
- [Règle d'or](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)

16. [Chapitre 16 - Pull requests \(PR\)](#)

- [Scenario type](#)
- [Pourquoi pas merger en silence ?](#)
- [Bonne description](#)
- [But](#)
- [Changes](#)
- [Test](#)
- [Draft PR](#)
- [Review](#)
- [Merge buttons](#)
- [A toi](#)
- [En vrai, sur le terrain](#)
- [Mini defi](#)

17. [Chapitre 17 - Atelier : lire les messages Git](#)

- ["nothing to commit, working tree clean"](#)
- ["untracked files"](#)
- ["Your branch is ahead of origin/main"](#)
- ["Your branch is behind"](#)
- ["rejected ... fetch first"](#)
- ["merge conflict"](#)
- ["Permission denied" / auth](#)

- [Methode](#)
- [Exercices](#)
- [Check](#)
- [Traduction rapide](#)

18. [Chapitre 18 - Atelier : workflow d'equipe \(simule\)](#)

- [Idee](#)
- [Tour de jeu](#)
- [Regles d'equipe simples](#)
- [force push ?](#)
- [Protection de branche](#)
- [A toi](#)
- [Mini defi](#)
- [Mini charte \(copie-colle\)](#)

19. [Chapitre 19 - Atelier : reparer sans paniquer](#)

- [Cas 1 - Mauvais fichier stage](#)
- [Cas 2 - Mauvais message \(pas encore push\)](#)
- [Cas 3 - Commit en trop \(pas push\)](#)
- [Cas 4 - Commit deja push \(equipe\)](#)
- [Cas 5 - Mauvaise branche](#)
- [Cas 6 - Conflit qui te depasse](#)
- [Checklist anti-stress](#)
- [Exercice final atelier](#)
- [En vrai](#)

20. [Quiz final - Teste-toi !](#)

- [Reponses](#)
- [Score](#)

21. [Bravo.](#)

- [Ce que tu sais faire](#)
- [Mission finale](#)
- [La suite \(quand tu voudras\)](#)
- [Rituel des 2 minutes](#)

Chapitre 1 - Salut, c'est quoi Git ?



Git garde l'historique de ton projet, version apres version.

Git, c'est un outil pour garder l'historique de ton projet.

Chaque "photo" s'appelle un commit.

Tu modifies des fichiers.

Tu dis a Git : "garde cette version".

Plus tard, tu peux revenir en arriere. Ou comparer. Ou travailler a plusieurs.

A quoi ca sert ?

Tu ne perds plus ton travail aussi facilement. Tu vois qui a change quoi, et quand. Tu peux essayer une idee sur une branche sans casser le reste. Et tu peux collaborer avec GitHub, GitLab, etc.

Git vs GitHub

Git, c'est le logiciel sur ton ordi. **GitHub**, c'est un site qui heberge des depots Git en ligne. Tu peux utiliser Git tout seul, sans GitHub. GitHub devient utile pour partager, sauvegarder au loin, et faire des pull requests.

Image mentale

Ton projet = un album photo.

Chaque commit = une photo datee + un petit titre.

Les branches = des albums paralleles.

Ce que tu vas savoir faire

Tu vas installer Git et configurer ton nom / email. Tu creeras un depot avec `git init`. Tu manipuleras `add`, `commit`, `status` et `log`. Tu feras des branches, des fusions et des conflits. Tu ecriras un `.gitignore`. Tu cloneras, pousseras et tireras depuis GitHub. Tu tapoteras stash et les annulations simples. Et tu liras / ouvriras une pull request.

Comment lire ce livre

Lis. Tape les commandes dans un vrai dossier de test. Regarde ce que Git repond. Casse volontairement, puis repare.

Git s'apprend avec les mains.

Pas seulement avec les yeux.

Mythe a casser

"Git, c'est trop complique pour un debutant."

Non.

Les 5 commandes du quotidien suffisent deja a etre utile.

Le reste vient apres.

Allez. On installe.

En vrai, sur le terrain

Dis a voix haute : "Je veux une machine a photos de mon code."

C'est Git.

Mini defi

Ecris 3 peurs / doutes sur Git (ex : "j'ai peur d'effacer").

On y repondra dans le livre.

Chapitre 2 - Installer Git

Windows

1. Va sur git-scm.com
2. Telecharge l'installateur
3. Suis l'assistant (les options par default vont bien au debut)
4. Ouvre un terminal et tape :

```
git --version
```

Si tu vois un numero, c'est bon.

Premiere config (obligatoire)

Git a besoin de savoir qui tu es pour signer tes commits :

```
git config --global user.name "Ton Prenom"  
git config --global user.email "ton@email.com"
```

Verifie :

```
git config --global --list
```

Utilise un email que tu assumeras (souvent celui de ton compte GitHub).

Ou taper les commandes ?

Tu peux utiliser le terminal Windows (PowerShell), Git Bash (installe avec Git), ou le terminal integre de VS Code. Les commandes de ce livre marchent dans ces environnements. Sur PowerShell, c'est le meme

```
git ...
```

Editeur de message

Parfois Git ouvre un editeur pour ecrire un message de commit.

Au debut, reste sur les messages en une ligne avec `-m "..."` .

Plus simple.

Aide integree

```
git help  
git help commit
```

Ou :

```
git commit -h
```

A toi

Lance `git --version`. Configure `user.name` et `user.email`. Affiche la config.

Erreur classique

Tu commits sans avoir configure le nom/email.

Git rale. Ou pire : il utilise une fausse identite.

Configure tout de suite.

En vrai, sur le terrain

Ouvre VS Code. Ouvre le terminal. Tape `git --version`.

Si ca marche la, tu es pret pour la suite.

Mini defi

Cree un dossier `mes-tests-git` sur ton disque.

C'est ton terrain de jeu pour tout le livre.

Chapitre 3 - Ton premier depot

Un depot (repository) = un projet suivi par Git.

Creer le dossier

```
mkdir mon-carnet  
cd mon-carnet
```

Initialiser Git

```
git init
```

Git cree un dossier cache `.git`.

Ne le touche pas a la main. C'est la memoire de Git.

Premier fichier

Cree `readme.txt` avec :

```
Mon carnet de tests Git
```

Voir l'etat

```
git status
```

Git dit souvent quels fichiers sont non suivis (untracked), modifies, ou prêts a commit. Lis `status` souvent. C'est ton tableau de bord.

Zones a retenir

Le **dossier de travail**, ce sont tes fichiers normaux. L'**index / staging**, c'est ce que tu prepares pour la photo. Le **depot**, c'est l'historique des commits. On detaille `add` / `commit` au chapitre suivant.

git init dans un projet existant

Tu peux lancer `git init` dans un dossier qui a deja des fichiers.

Puis tu choisis quoi ajouter.

A toi

Fais `git init` dans `mes-tests-git` (ou `mon-carnet`). Cree un fichier texte. Lance `git status` et lis le resultat a voix haute.

Erreur classique

Tu fais `git init` dans le mauvais dossier (ex : ton dossier utilisateur entier).

Verifie avec `cd` et `pwd` (ou `cd` sous PowerShell) avant.

En vrai, sur le terrain

Ouvre l'explorateur. Active "elements caches".

Tu dois voir `.git` apparaitre apres `init`.

Mini defi

Ajoute un second fichier `notes.txt`.

Relance `status`. Compare avec avant.

Trois zones, encore une fois

Imagine une chaine. Tu ecris dans tes fichiers (zone de travail). Tu choisis quoi photographier (`add` = index). Tu valides la photo (`commit` = historique).

Beaucoup de debutants confondent 2 et 3.

Si `status` dit "Changes to be committed", c'est pret pour la photo.

Si ca dit "Changes not staged", il manque un `add`.

Chapitre 4 - add et commit (la photo)



add prepare. commit photo. Une etape claire.

Le duo du quotidien : `git add` pour preparer, puis `git commit` pour prendre la photo.

Ajouter un fichier

```
git add readme.txt
git status
```

Le fichier passe en "staged" (index).

Pret pour le commit.

Ajouter plusieurs fichiers

```
git add .
```

Le point = "tout ce qui a change ici".

Pratique. Mais regarde `status` avant : tu ne veux pas ajouter un secret par accident.

Commit

```
git commit -m "Premier commit : readme de base"
```

`-m` = message en une ligne.

Le message doit dire **pourquoi** / **quoi** de utile. Pas juste "update".

Voir que ca a marche

```
git status
git log --oneline
```

`status` devrait dire que rien ne reste a commit.

`log` montre ta photo.

Modifier encore

Change `readme.txt`, puis :

```
git status
git add readme.txt
git commit -m "Clarifier le titre du carnet"
```

Chaque commit = une etape claire.

Schema mental

```
modifier fichiers
-> git add
-> git commit
-> historique +1
```

Amend (aperçu, avec prudence)

Tu viens de commit et tu as oublie un fichier ?

```
git add fichier_oublie.txt
git commit --amend --no-edit
```

Ca refait le dernier commit.

Ne l'utilise pas sur un commit deja pousse si tu travailles en equipe (sauf si tu sais pourquoi).

A toi

Fais 2 commits : d'abord la creation du readme, puis l'ajout d'une ligne.

En vrai, sur le terrain

Apres chaque commit, `git log --oneline`.

Tu dois voir tes messages s'empiler.

Mini defi

Cree `todo.txt` avec 3 taches.

Commit. Coche une tache. Commit encore.

Chapitre 5 - Lire l'historique

L'interet de Git, c'est de pouvoir regarder en arriere sans paniquer.

log simple

```
git log
```

Tu vois hash, auteur, date, message.

Quitte avec `q` si c'est une vue longue.

log compact

```
git log --oneline
```

Une ligne par commit. Ideal au debut.

Voir un fichier a travers le temps

```
git log --oneline -- readme.txt
```

diff : qu'est-ce qui a change ?

Avant de commit :

```
git diff
```

Ca montre les modifications non stagees.

Entre staging et dernier commit :

```
git diff --staged
```

Entre deux commits :

```
git diff HEAD~1 HEAD
```

`HEAD` = ou tu es maintenant.

`HEAD~1` = le commit d'avant.

show

```
git show HEAD
```

Detail du dernier commit.

Checkout d'un fichier (aperçu)

Pour jeter les modifs non commit d'un fichier :

```
git restore readme.txt
```

(Anciennement `git checkout -- readme.txt`.)

Attention : tu perds les changements non sauves dans un commit.

A toi

Fais un petit changement. Lance `git diff`. Puis `add` + `commit`. Termine avec `git log --oneline`.

En vrai, sur le terrain

Ouvre `git log --oneline --graph --all` une fois.

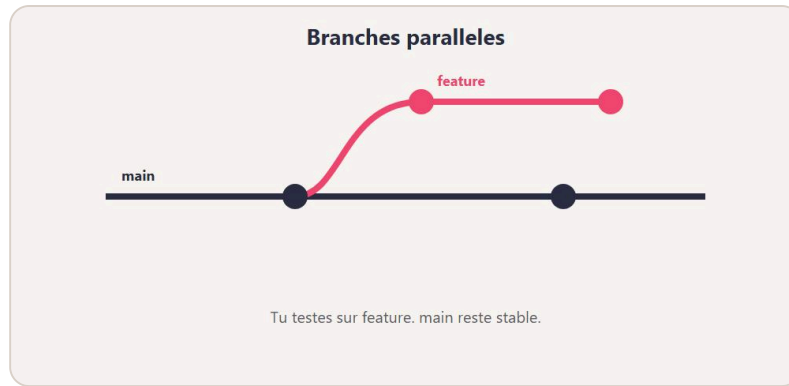
Meme sur un petit projet, tu vois la forme de l'histoire.

Mini defi

Cree 3 commits avec des messages tres clairs.

Demande a quelqu'un de lire ton `log --oneline` : est-ce comprehensible ?

Chapitre 6 - Les branches



Une branche = une piste parallele pour tester sans casser.

Une branche = une piste parallele.

Tu experimentes sans casser la version stable.

Voir les branches

```
git branch
```

Souvent tu as `main` (ou `master` sur de vieux depots).

Creer et changer de branche

```
git branch idee-couleurs
git switch idee-couleurs
```

Ou en une commande :

```
git switch -c idee-couleurs
```

(`git checkout -b ...` existe encore. `switch` est plus clair.)

Travailler sur la branche

Modifie un fichier. Commit.

```
git log --oneline
```

Ces commits sont sur `idee-couleurs`.

`main` n'a pas encore ces changements.

Revenir sur main

```
git switch main
```

Tes fichiers "reviennent" a l'etat de `main` .

Magique la premiere fois. Normal ensuite.

A quoi ca sert vraiment ?

Tu lances une feature en cours. Tu corriges en urgence sur `main` pendant que tu experimentes ailleurs. Ou tu testes un essai risque, puis tu jettes la branche si ca ne marche pas.

Renommer / supprimer

```
git branch -m ancien-nom nouveau-nom
git branch -d idee-couleurs
```

`-d` refuse s'il reste des commits non fusionnes.

`-D` force (prudent).

A toi

Cree une branche `essai` . Ajoute un fichier `essai.txt` et commit. Reviens sur `main` . Verifie que `essai.txt` n'est plus la (normal).

En vrai, sur le terrain

Avant chaque experience : nouvelle branche.

Reflexe pro, meme en solo.

Mini defi

Deux branches : `page-a` et `page-b` .

Un commit different sur chacune. Note ce que `log` montre sur chaque.

Chapitre 7 - Fusionner (merge)

Tu as travaillé sur une branche. Tu veux ramener le travail dans `main`.

Scenario

```
git switch main
git merge idee-couleurs
```

Git essaie de combiner les historiques.

Si tout va bien

Git crée souvent un "merge commit" (ou avance juste le pointeur : fast-forward).

Tu vois le résultat dans les fichiers + dans `log`.

Fast-forward, c'est quoi ?

Si `main` n'a pas bougé pendant ton travail, Git peut juste avancer.

Pas de conflit. Simple.

Merge avec message

Parfois Git ouvre un éditeur.

Tu peux aussi :

```
git merge idee-couleurs -m "Fusion idee couleurs"
```

Après le merge

```
git log --oneline --graph
git branch -d idee-couleurs
```

Tu peux supprimer la branche locale une fois fusionnée.

Quand ne pas merger directement ?

Sur GitHub, on passe souvent par une **pull request** (chapitre 16).

En local / solo, `merge` suffit pour apprendre.

A toi

Crée une branche `ajout-note`. Commit un fichier. Merge dans `main`. Regarde `log --graph --oneline`.

En vrai, sur le terrain

Fais un dessin : `main` et `feature` qui se rejoignent.

Le merge, c'est le point de jonction.

Mini defi

Deux branches avec chacune un fichier different.

Merge les deux dans `main`. Les deux fichiers doivent etre presents.

Merge vs rebase (aperçu)

`merge` : garde l'historique tel quel, ajoute souvent un commit de jonction.

`rebase` : rejoue tes commits "par-dessus" une autre branche. Historique plus lineaire, mais plus piegeux.

Pour ce livre : maitrise `merge` d'abord.

Le rebase, tu le croiseras plus tard (avec prudence, surtout sur branches partagees).

Chapitre 8 - Les conflits

Un conflit = Git ne sait pas choisir.

Deux versions du meme endroit. Il te demande d'arbitrer.

Comment ca arrive

Sur `main`, tu modifies la ligne 1 de `readme.txt`. Sur une branche, tu modifies aussi la ligne 1. Tu merges : boum, conflit.

A quoi ca ressemble

Dans le fichier :

```
<<<<<< HEAD
version de main
=====
version de la branche
>>>>>> idee-couleurs
```

Tu gardes ce que tu veux. Tu enlèves les marqueurs `<<<<<<` etc.

Resolving

Ouvre le fichier. Corrige a la main. Fais `git add fichier_corrige.txt`, puis `git commit` pour conclure le merge. Ou, si ton outil propose "accepter actuel / entrant / les deux", OK - mais comprends ce que tu acceptes.

Annuler un merge en cours

Si tu es perdu :

```
git merge --abort
```

Retour avant le merge. Ouf.

Eviter les conflits (un peu)

Fais des petits commits. Parle-toi en equipe. Merge / rebase souvent (pas attendre 3 semaines). Evite que deux personnes editent la meme zone sans coordination.

A toi

Provoque un conflit volontairement sur `readme.txt`.

Resols-le. Termine le merge.

En vrai, sur le terrain

Un conflit n'est pas un echec.

C'est Git qui dit : "decide humain".

Mini defi

Note 3 regles perso pour limiter les conflits dans ton prochain projet.

Chapitre 9 - .gitignore

Certains fichiers ne doivent **jamais** aller dans Git.

Secrets, caches, gros binaires, dossiers de dependances...

Creer le fichier

A la racine du projet, cree `.gitignore` :

```
# secrets
.env
*.key

# python
__pycache__/
.venv/

# node
node_modules/

# OS / editeur
.DS_Store
Thumbs.db
.idea/
.vscode/
```

Adapte a ton projet. Ce n'est pas une liste magique universelle.

Verifier

```
git status
```

Les fichiers ignores ne doivent plus apparaitre comme "a ajouter".

Deja suivi par erreur ?

Si tu as deja `add` un fichier sensible :

```
git rm --cached fichier.env
```

Ca l'enleve de l'index, pas de ton disque.

Puis commit. Et ajoute-le au `.gitignore`.

Si le secret est deja sur GitHub : **change le secret** (mot de passe, cle).

L'enlever de l'historique est un autre niveau.

Templates

GitHub propose des `.gitignore` tout faits (Python, Node...).

Tu peux t'en inspirer.

A toi

Ajoute un `.gitignore` a ton carnet de tests.

Cree un faux `.env` avec `SECRET=demo`.

Verifie que Git l'ignore.

En vrai, sur le terrain

Avant le premier push public : relis `git status`.

Cherche mots de passe, cles, dumps, photos perso.

Mini defi

Ecris un `.gitignore` pour un mini site (HTML/CSS/JS) avec un dossier `dist/` a ignorer.

Patterns utiles

```
*.log
tmp/
build/
dist/
*.pdf
!docs/manuel.pdf
```

`!` peut re-inclure une exception.

Commence simple. Complexifie seulement si besoin.

Chapitre 10 - GitHub, c'est quoi ?

GitHub heberge tes depots Git sur internet.

Sauvegarde distante + collaboration + issues + pull requests.

Compte

Cree un compte sur github.com. Confirme l'email. Optionnel mais recommande : active la double authentication.

Nouveau depot en ligne

Sur GitHub : **New repository**. Choisis un nom, par exemple `mon-carnet`. Public ou prive. Tu peux ne **pas** cocher "Add a README" si tu as deja un depot local.

Lier ton depot local

Sur la page du depot vide, GitHub montre des commandes.

En general :

```
git remote add origin https://github.com/TON_COMPTE/mon-carnet.git
git branch -M main
git push -u origin main
```

`origin` = surnom classique de ton remote principal.

Voir les remotes

```
git remote -v
```

SSH ou HTTPS ?

HTTPS est simple au debut (parfois un token). SSH utilise des cles et devient tres confortable au quotidien. Les deux marchent. Choisis-en un et avance.

README.md

Sur GitHub, `README.md` s'affiche en vitrine.

Ecris 5 lignes : but du projet + comment lancer.

A toi

Cree un depot GitHub de test (prive OK). Branche `origin`. Fais `push` de ta branche `main`.

En vrai, sur le terrain

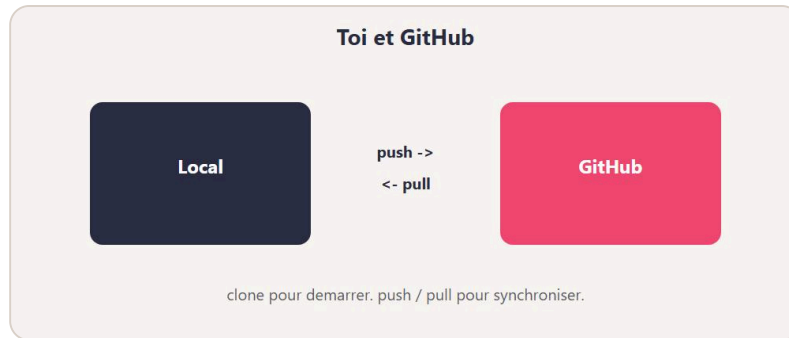
Apres le push, rafraichis la page GitHub.

Tu dois voir tes fichiers. Soulagement.

Mini defi

Ajoute un vrai `README.md` clair. Commit. Push. Verifie le rendu.

Chapitre 11 - clone, push, pull



clone / pull / push : le dialogue avec GitHub.

Le dialogue avec le serveur.

clone

Recuperer un depot existant :

```
git clone https://github.com/TON_COMPTE/mon-carnet.git
cd mon-carnet
```

`clone` = copie + lien `origin` deja configure.

push

Envoyer tes commits locaux vers le remote :

```
git push
```

La premiere fois (si pas de `-u`) :

```
git push -u origin main
```

pull

Recuperer les commits distants et les fusionner chez toi :

```
git pull
```

Reflexe avant de recommencer a coder le matin : `git pull`.

fetch (aperçu)

```
git fetch
```

`fetch` telecharge les infos sans fusionner tout de suite.

`pull` $\approx$ `fetch` + merge (en simplifiant).

Schema

```
toi (local) --push--> GitHub  
toi (local) <--pull-- GitHub
```

Erreurs frequentes

`rejected` : le remote a des commits que tu n'as pas -> `pull` d'abord. Auth ratee : mauvais token / session.
Mauvaise branche : tu pushes `main` alors que tu es ailleurs.

A toi

Modifie un fichier, commit, puis push. Change quelque chose sur GitHub via l'interface (petit edit). Fais `git pull` en local.

En vrai, sur le terrain

Sur un projet a deux : toujours `pull` avant de pousser.

Ca diminue les conflits.

Mini defi

Clone un petit depot public open source (lecture seule).

Explore `log --oneline`. Ne push rien.

Chapitre 12 - De bons messages de commit

Un bon message sauve ton futur toi.

Et ton equipe.

Regle simple

La premiere ligne resume court (50-72 caracteres ideaux). Prefere un verbe a l'imperatif : "Ajouter", "Corriger", "Clarifier". Explique le **pourquoi** si ce n'est pas evident.

Oui

```
Corriger le total TTC (oubli de la TVA)
Ajouter la page contact
Retirer le fichier .env du suivi
```

Non

```
update
fix
wip
asdf
final final 2
```

Corps optionnel

```
git commit -m "Corriger le calcul du panier" -m "Le total ignorait les codes promo. Desormais appliques avant TVA."
```

Conventional Commits (aperçu)

Tu verras parfois :

```
feat: ajouter export PDF
fix: corriger lien casse du sommaire
docs: preciser l'install Windows
```

Utile en equipe. Pas obligatoire pour apprendre.

Une idee = un commit ?

Idealement : un commit = une intention.

Pas "j'ai change 40 fichiers sans rapport".

Pas non plus 40 micro-commits illegibles.

Le juste milieu vient avec l'experience.

A toi

Reecris ces messages pourris en bons messages : `update`, `fix stuff`, et `aaa`.

En vrai, sur le terrain

Lis ton `git log --oneline`.

Si tu ne comprends plus un message d'hier, renomme ta facon d'ecrire.

Mini defi

Fais 3 commits sur ton carnet avec des messages "pro".

Exemple de fil propre

```
Initialiser le projet  
Ajouter la page d'accueil  
Corriger le lien du menu mobile  
Ignorer le fichier .env  
Documenter l'installation dans le README
```

Tu peux lire ca dans 10 secondes et comprendre l'histoire.

C'est ca, le but.

Chapitre 13 - Mini-projet : carnet versionne

On enchaîne le vrai workflow du quotidien.

But

Un petit projet `carnet-git` avec un `README.md`, un `notes.md`, un historique propre, une branche de feature, et un push GitHub.

Étapes

1. Local

```
mkdir carnet-git  
cd carnet-git  
git init
```

Ecris un README (but + auteur).

Commit: Initialiser le carnet

2. Notes

Ajoute `notes.md` avec trois idées courtes.

Commit: Ajouter les premières notes

3. Branche

```
git switch -c feature/note-git
```

Ajoute une note sur Git.

Commit.

Reviens sur `main` et merge.

4. Ignore

Ajoute `.gitignore` avec `.env` et un faux secret local.

Commit.

5. GitHub

Crée le dépôt distant.

```
remote add + push -u origin main
```

6. Preuve

Copie l'URL GitHub dans ton `README.md`.

Commit + push.

Criteres de reussite

Tu as au moins 4 commits lisibles, une branche fusionnee, un `.gitignore`, et un projet visible sur GitHub (prive OK).

Variante avancee

Ouvre une issue "Ameliorer le README". Cree une branche depuis cette idee. Ouvre une pull request (chapitre 16).

En vrai, sur le terrain

Chronometre-toi. Vise moins de 20 minutes une fois a l'aise.

Mini defi

Invite un ami en lecture seule sur le depot (s'il a un compte).

Chapitre 14 - Ce qu'il faut retenir (bases)

Boucle quotidienne

```
status -> add -> commit -> (push)
pull souvent si collab
```

Commandes vitales

Tu démarres avec `git init` ou `git clone`. Tu regardes l'état avec `git status`. Tu prépares avec `git add`, puis tu photographies avec `git commit`. Pour l'histoire : `git log --oneline`. Pour les pistes parallèles : `git switch` et `git branch`, puis `git merge`. Avec le distant : `git pull`, `git push`, et `git remote -v` pour vérifier le lien.

Zones

Travail -> index (`add`) -> historique (`commit`) -> remote (`push`)

Habitudes

Fais `status` avant et après. Ecris des messages clairs. Mets un `.gitignore` tôt. Préfère les petites branches. Ne commit jamais un secret.

Erreurs classiques

On commit parfois sur la mauvaise branche. Un push peut être refusé si le remote est en avance. Un conflit laisse des `<<<<<<` si tu oublies de nettoyer. Et `git add .` sans regarder, c'est le classique des secrets qui passent.

Suite immédiate

Deux chapitres un cran au-dessus : stash + annuler proprement, puis les pull requests. Puis ateliers + quiz.

Mini check

Sans notes : écris la boucle `add/commit/push` et à quoi sert une branche.

Aide-mémoire express

Pour l'état : `git status`. Pour préparer : `git add`. Pour la photo : `git commit -m "..."`. Pour l'histoire : `git log --oneline`. Pour une branche : `git switch -c nom`. Pour fusionner : `git merge nom`. Pour envoyer : `git push`. Pour recevoir : `git pull`.

Imprime cette page mentalement.

Chapitre 15 - Stash et annuler (avec prudence)

Parfois tu dois changer de branche alors que tu as des modifs pas pretes.

stash

```
git stash
git switch autre-branche
# ...
git switch -
git stash pop
```

`stash` = met de cote.

`stash pop` = recupere (et enleve de la pile).

Voir la pile :

```
git stash list
```

restore (annuler des modifs locales)

```
git restore fichier.txt
```

Revient a la derniere version commit (pour ce fichier).

Les changements non commits sont perdus.

restore --staged

```
git restore --staged fichier.txt
```

Retire du staging, garde les modifs dans le dossier.

reset --soft / --mixed (aperçu)

```
git reset --soft HEAD~1
```

Annule le dernier commit, garde les changements stages.

```
git reset HEAD~1
```

Annule le commit, garde les changements non stages.

reset --hard (danger)

```
git reset --hard HEAD~1
```

Revient en arriere et jette les modifs.

Puissant. Dangereux. Pas sur un commit déjà partagé sans discussion.

revert (plus sur en équipe)

```
git revert HEAD
```

Crée un **nouveau** commit qui annule le précédent.

L'historique reste honnête. Préfère ça sur `main` partagé.

Règle d'or

En local, pas pousser : reset possible. Déjà poussé / équipe : revert (ou discussion).

A toi

Modifie un fichier. Fais `stash`. Vérifie que `status` est propre. Puis `stash pop`.

En vrai, sur le terrain

Avant un `--hard`, respire.

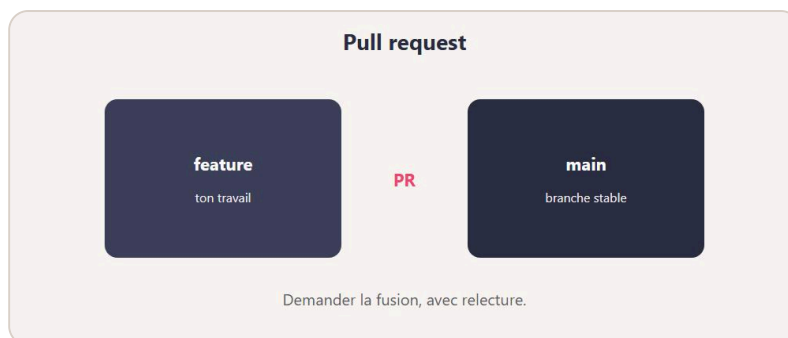
Copie le dossier ailleurs si tu as un doute.

Mini défi

Fais un commit "oops".

Annule-le avec `revert`. Regarde le `log`.

Chapitre 16 - Pull requests (PR)



Une pull request, c'est demander de fusionner ton travail.

Une pull request = "peux-tu fusionner mon travail dans la branche principale ?"

C'est le coeur de la collab sur GitHub.

Scenario type

Tu crées une branche avec `git switch -c feature/login`, tu fais tes commits locaux, puis `git push -u origin feature/login`. Sur GitHub, tu cliques **Compare & pull request**. Tu remplis titre + description. Quelqu'un review. On merge sur GitHub. En local, tu reviens avec `git switch main` puis `git pull`.

Pourquoi pas merger en silence ?

Ca permet la relecture, la discussion, les tests automatiques (CI), et un historique plus clair. Même en solo, une PR t'oblige à résumer ton travail. C'est sain.

Bonne description

```
## But
Ajouter un formulaire de contact.

## Changes
- page contact.html
- validation email basique

## Test
Ouvrir /contact et envoyer un faux message.
```

Draft PR

Tu peux ouvrir une PR en brouillon.

Utile pour montrer l'avancement sans demander le merge tout de suite.

Review

En review, on commente des lignes.

Tu pousses des commits sur la même branche : la PR se met à jour.

Merge buttons

Tu verras souvent **Create a merge commit**, **Squash and merge** (souvent propre pour petites features), et **Rebase and merge**. Demande la convention de ton equipe.

A toi

Cree une branche avec 1 commit. Push. Ouvre une PR vers `main`. Merge-la toi-meme. Puis `git pull` en local.

En vrai, sur le terrain

Lis 2 PR open source.

Tu verras le meme format partout.

Mini defi

Ecris un modele de description PR dans ton README (section Contributing).

Chapitre 17 - Atelier : lire les messages Git

Git parle anglais souvent. On traduit.

"nothing to commit, working tree clean"

Rien à photographier. Tout est déjà commit. OK.

"untracked files"

Fichiers pas encore suivis. `add` si tu les veux. Sinon `.gitignore`.

"Your branch is ahead of origin/main"

Tu as des commits locaux non poussés. `git push`.

"Your branch is behind"

Le remote a avancé. `git pull` avant de continuer.

"rejected ... fetch first"

Push refuse. Tire d'abord. Résous. Repush.

"merge conflict"

Ouvre les fichiers marqués. Nettoie. `add`. commit de merge.

"Permission denied" / auth

Problème d'identifiants (HTTPS token, SSH key, droits du dépôt).

Méthode

Lis d'abord la **dernière** partie du message. Ne panique pas sur le pavé rouge. Lance `git status` pour savoir où tu es. Une action à la fois.

Exercices

Provoque un push reject (pull manquant) et répare. Provoque un conflit et le résout. Ajoute un secret au mauvais moment, puis retire-le avec `rm --cached`.

Check

Tu sais expliquer 4 messages courants sans google.

Traduction rapide

Si le working tree est clean, rien à commit. Untracked : add ou ignore. Ahead : push. Behind : pull. Rejected : pull puis push. Conflict : éditer + add + commit.

Garde ces réflexions sous la main la première semaine.

Chapitre 18 - Atelier : workflow d'equipe (simule)

Meme seul, tu peux simuler deux personnes.

Idee

Deux dossiers locaux : `alice/carnet` et `bob/carnet`. Tous deux clones du meme depot GitHub.

Tour de jeu

Alice cree une branche `feature/alice`, commit, push, PR. Bob fait `pull` sur `main` apres le merge. Bob cree `feature/bob` sur une autre zone de fichier, puis sa PR. Alice review (meme si c'est toi sous un autre navigateur).

Regles d'equipe simples

Pas de commit direct sur `main` : branche + PR. Messages clairs. PR petites. `pull` le matin. Ne jamais forcer (`push --force`) sur `main`.

force push ?

```
git push --force
```

A bannir sur les branches partagees tant que tu ne maitrises pas.

Ca reecrit l'histoire distante. Ca peut effacer le travail des autres.

Protection de branche

Sur GitHub : Settings -> Branches -> proteger `main`.

Exige une PR. C'est une ceinture de securite.

A toi

Joue Alice et Bob sur un depot de test.

2 PR mergees. Historique lisible.

Mini defi

Ecris un fichier `CONTRIBUTING.md` de 10 lignes pour ton futur projet.

Mini charte (copie-colle)

Personne ne pousse en force sur `main`. Toute feature passe par une PR. On review au moins 1 fois (meme soi-meme en solo). On ne commit pas `.env`. On ecrit des messages qui se lisent.

Colle ca dans `CONTRIBUTING.md`.

Chapitre 19 - Atelier : reparer sans paniquer

Cas 1 - Mauvais fichier stage

```
git restore --staged mauvais.txt
```

Cas 2 - Mauvais message (pas encore push)

```
git commit --amend -m "Meilleur message"
```

Cas 3 - Commit en trop (pas push)

```
git reset --soft HEAD~1
```

Cas 4 - Commit deja push (equipe)

```
git revert HEAD  
git push
```

Cas 5 - Mauvaise branche

Tu as commit sur `main` au lieu de `feature` :

```
git switch -c feature/oubli  
git switch main  
git reset --hard HEAD~1  
git switch feature/oubli
```

Seulement si le commit n'est pas utile sur `main` et pas partage.

Sinon : demande conseil / fais une PR depuis ce commit autrement.

Cas 6 - Conflit qui te depasse

```
git merge --abort
```

Repars. Demande de l'aide. Reessaye plus tard.

Checklist anti-stress

Commence par `git status`, puis `git log --oneline -5`. Demande-toi : est-ce deja pousse ? Suis-je seul sur la branche ? Alors seulement : restore / reset / revert.

Exercice final atelier

Casse volontairement (mauvais add, mauvais message, petit conflit).

Repare avec la methode ci-dessus.

Ecris dans `notes.md` ce que tu as fait.

En vrai

Les seniors aussi cassent.

La difference : ils savent lire `status` et choisir un outil sur.

Quiz final - Teste-toi !

Q1. Git sert surtout a :

- A) Dessiner des maquettes
- B) Garder l'historique des versions
- C) Remplacer un antivirus

Q2. GitHub, c'est :

- A) Un hebergeur de depots Git (+ collab)
- B) Un langage
- C) Un fichier `.gitignore`

Q3. `git init` :

- A) Envoie sur internet
- B) Cree un depot local
- C) Supprime l'historique

Q4. Avant `commit`, on fait souvent :

- A) `git add`
- B) `git push`
- C) `git clone`

Q5. `git status` sert a :

- A) Voir l'etat du dossier / index
- B) Fusionner
- C) Creer une PR

Q6. Une branche permet de :

- A) Compresser les images
- B) Travailler en parallele sans casser main
- C) Effacer GitHub

Q7. `git merge` :

- A) Clone un depot
- B) Fusionne des historiques / branches
- C) Ignore les fichiers

Q8. Un conflit veut dire :

- A) Git a plante pour toujours
- B) Deux versions du meme endroit a arbitrer
- C) Le disque est plein

Q9. `.gitignore` sert a :

- A) Decoruer le README
- B) Exclure des fichiers du suivi
- C) Accelerer internet

Q10. `git push` :

- A) Envoie tes commits vers le remote
- B) Telecharge seulement
- C) Cree une branche locale

Q11. `git pull` :

- A) Supprime origin
- B) Recupere (et integre) les commits distants
- C) Remplace `commit`

Q12. Un bon message de commit :

- A) `update`
- B) `Corriger le lien casse du menu`
- C) `asdf`

Q13. `git stash` :

- A) Met des modifs de cote temporairement
- B) Pousse sur GitHub
- C) Cree un remote

Q14. Sur un commit deja partage en equipe, prefere :

- A) `reset --hard` silencieux
- B) `git revert`
- C) Effacer le depot

Q15. Une pull request, c'est :

- A) Demander a fusionner une branche (souvent via review)
- B) Un virus
- C) Un type de conflit

Q16. `git clone` :

- A) Copie un depot distant en local
- B) Efface main
- C) Remplace status

Q17. `git log --oneline` :

- A) Affiche l'historique compact
- B) Installe Git
- C) Ouvre GitHub

Q18. Si push est rejected car remote en avance :

- A) Ignorer
- B) `git pull` puis reparer eventuellement, puis push
- C) Formater le PC

Reponses

1 B, 2 A, 3 B, 4 A, 5 A, 6 B, 7 B, 8 B, 9 B, 10 A,
11 B, 12 B, 13 A, 14 B, 15 A, 16 A, 17 A, 18 B

Score

16-18 : nickel. 12-15 : bien, relis conflits + undo + PR. 8-11 : refais 4, 6, 8, 11, 14. Moins de 8 : refais le mini-projet tranquillement.

FINAL

Bravo.



Bravo. Tu as les bases de Git et GitHub.

Tu as fini les bases de Git et GitHub.

Pas "je connais toutes les options cachees".

Mais "je sais versionner, brancher, fusionner, pousser, et me sortir d'un conflit". C'est le coeur du metier.

Ce que tu sais faire

Tu sais installer et configurer Git. Tu utilises `add`, `commit`, `log` et `status`. Tu geres branches, merge et conflits. Tu ecris un `.gitignore`. Tu manipules remote, clone, push et pull. Tu te debrouilles avec stash et les annulations raisonnables. Et tu fais des pull requests.

Mission finale

Sur un vrai mini projet (meme un site de 2 pages), cree un depot GitHub. Travaille sur une branche feature. Merge via une PR. Ecris un README clair. Verifie qu'aucun secret n'est dans l'historique.

La suite (quand tu voudras)

Tu pourras regarder les hooks / pre-commit, le rebase interactif (avec prudence), les monorepos, GitHub Actions (CI), et des conventions d'equipe plus fines.

Mais la, respire.

Encore bravo.

Ton code a maintenant une memoire. Et toi aussi.

Rituel des 2 minutes

Avant chaque session :

```
git status
git pull
```

Apres chaque morceau utile :

```
git status
git add .
git commit -m "...
git push
```

Regarde toujours `status` avant `add .`.

Les habitudes battent la memoire.

Tu as les briques. Maintenant, construis.